



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 08

Team No.: 05

Project Title: Linux Process Creation, Execution and Termination System

Team Members

Roll Number	Student Name	Signature
2520030276	V Sameer Reddy	
2520030306	Ch Sahasra Gayathri	

1. Abstract

Process management is a fundamental responsibility of an operating system. Every program running in a Linux system is executed as a process, and the operating system provides mechanisms to create, execute, synchronize, monitor, and terminate these processes. This project, **Linux Process Creation, Execution and Termination System**, aims to provide a practical demonstration of the complete lifecycle of a process using Linux/POSIX system programming concepts.

The proposed system will be implemented using the C programming language in a Linux environment. A menu-driven application will allow users to create child processes, execute programs or commands, display process information, synchronize parent and child processes, and terminate processes in a controlled manner. The project will make use of Linux/POSIX system calls such as `fork()`, `exec()`, `getpid()`, `getppid()`, `wait()`, `waitpid()`, `exit()`, and optionally `kill()`.

The project provides hands-on understanding of parent-child process relationships and demonstrates how the operating system manages process execution and termination. The final system will serve as an educational tool for understanding practical Linux process management.

2. Problem Statement

Process creation, execution, synchronization, and termination are essential functions performed by an operating system. Although these concepts are studied theoretically, students may not fully understand

how processes interact with each other and how the Linux operating system manages their complete lifecycle.

There is a need for a practical system that demonstrates how a parent process creates a child process, how the child executes a program, how the parent waits for the child to complete, and how processes can be terminated in a controlled manner.

Therefore, this project aims to develop a Linux-based application that demonstrates the complete process lifecycle using Linux/POSIX system calls and provides practical understanding of process management.

3. Objectives

The objectives of the project are:

- To understand the process lifecycle in a Linux operating system.
- To create child processes using the `fork()` system call.
- To execute programs or commands using the `exec()` family of functions.
- To display process information such as Process ID and Parent Process ID.
- To demonstrate the relationship between parent and child processes.
- To synchronize processes using `wait()` and `waitpid()`.
- To obtain and display the termination status of child processes.
- To demonstrate controlled process termination using `exit()` and optionally `kill()`.
- To gain practical experience with Linux/POSIX system programming.

4. Proposed Methodology

The project will be implemented as a **menu-driven application in the C programming language** and executed in a Linux environment.

The working methodology will consist of the following steps:

1. The program starts and displays a menu containing different process management operations.
2. Based on the user's choice, the parent process performs the selected operation.
3. A child process is created using the `fork()` system call.
4. The program identifies the parent and child processes using `getpid()` and `getppid()`.
5. The child process can execute another program or Linux command using an `exec()` family function.
6. The parent process waits for the child process using `wait()` or `waitpid()`.
7. The termination status of the child process is collected and displayed.
8. The child process terminates using `exit()`. Controlled termination using `kill()` and signals may also be demonstrated.
9. The user can perform multiple operations until selecting the exit option.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
<code>fork()</code>	Creates a new child process from the parent process.
<code>exec()</code>	Executes a new program in the child process.
<code>getpid()</code>	Gets the current process ID.
<code>getppid()</code>	Gets the parent process ID.
<code>wait()</code>	Makes the parent wait for the child process.
<code>waitpid()</code>	Waits for a specific child process.
<code>exit()</code>	Terminates a process.
Process Management	Used to create, execute, monitor, and terminate processes.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030306	Ch Sahasra Gayathri	Implemented process creation using <code>fork()</code> and process identification using <code>getpid()</code> and <code>getppid()</code> .
2520030306	Ch Sahasra Gayathri	Implemented process execution using <code>exec()</code> and process synchronization using <code>wait()</code> / <code>waitpid()</code> .
2520030276	V Sameer Reddy	Implemented process termination using <code>exit()</code> , performed testing, and documented the results.
2520030276	V Sameer Reddy	Contributed to project design, debugging, documentation, and final presentation.

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Primary operating system environment for developing and running the project.

C Programming Language	Used to implement the project.
GCC Compiler	Used to compile the C source code.
POSIX/Linux System Calls	Used for process creation, execution, synchronization, and termination.
GNU Make	Used to automate the compilation and build process through a Makefile.
GDB	Used for debugging and analyzing the program if required.

8. Expected Outcome

The expected outcome of this project is a functional Linux-based application that demonstrates the complete lifecycle of processes.

The system will successfully allow users to:

- Create child processes.
- Display Process ID and Parent Process ID.
- Demonstrate the relationship between parent and child processes.
- Execute Linux programs or commands from child processes.
- Synchronize parent and child processes.
- Monitor the completion and exit status of child processes.
- Demonstrate controlled process termination.

9. GitHub Repository

GitHub Repository Link: https://github.com/sahasragayathri02/CinnamonBun_OSSP_Project

Repository Created: Yes

Faculty Name and Signature

Faculty Name: Dr.V.Muniraju Naidu

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____